

# Chapter 1

## L1 - Introduction

### 1.1 Why data management?

Modern software is data-driven: systems are useful only when data can be stored, shared, and queried correctly over time. This chapter builds that idea step by step, from raw data to DBMS architecture.

#### 1.1.1 What is data?

We first separate *raw facts* from *usable meaning*.

**Data.** Raw facts used as input for reasoning, discussion, and computation. Data can be useful, irrelevant, or redundant. It must be processed to become meaningful.

**Information.** Processed data with meaning in context. Information is relevant to a specific problem and actionable for decision-making.

Practical flow:

$$\text{Data} \xrightarrow{\text{organize} + \text{process}} \text{Information}$$

Information then supports higher-level outcomes such as **knowledge** and **wisdom**.

### 1.1.2 What is a database?

Once data has meaning, we need structure.

**Database.** A large, integrated, structured collection of data, usually modeling a real-world enterprise.

**Entity.** An object type we store data about (for example, a student or a course).

**Relationship.** A meaningful association between entities (for example, a student enrolls in a course).

*Example: University.* Entities are courses, students, and professors; relationships include enrollment and teaching.

### 1.1.3 What is a Database Management System (DBMS)?

A database alone is passive. We need software that manages access and correctness.

**DBMS.** A software system designed to store, manage, and facilitate access to databases.

A useful decomposition is:

$$\text{DBMS} = \text{interrelated data (database)} + \text{programs to access/manage it}$$

### 1.1.4 What does a DBMS do?

**Core role of a DBMS.** Provide efficient, reliable, convenient, and safe multi-user storage of and access to massive persistent data.

In practice, this means:

- failure protection against hardware/software faults, power loss, and malicious behavior,
- high throughput (thousands of queries/updates per second),
- high availability (24/7 operation),
- concurrency control for correct multi-user access,
- support for very large-scale data,
- persistence (data outlives programs),
- high-level declarative access with physical data independence.

**Persistent data.** Data that remains stored beyond the lifetime of a process or program execution.

**Concurrency control.** Mechanisms that coordinate simultaneous operations so outcomes remain correct.

### 1.1.5 Is your file system a DBMS?

This thought experiment shows why plain files are often insufficient for data-intensive multi-user applications.

#### Thought experiment 1: concurrent edits

Two collaborators edit and save the same file at nearly the same time. The natural question is: *whose changes survive?* Outcomes can be yours, your partner's, both, neither, or simply **unclear**.

#### Thought experiment 2: update during power loss

A file update is in progress and power fails. Which changes survive: all, none, only changes since last save, or again **unclear**?

The key issue is the missing contract: guarantees are unclear, which makes application logic fragile and hard to reason about.

## How do you write programs over a platform that promises “???”?

If correctness, durability, isolation, and recovery guarantees are not explicit, every application must re-implement them. **A DBMS exists to provide these guarantees systematically.**

### 1.1.6 Is the web a DBMS?

The web offers strong information retrieval, but retrieval is not equivalent to database management.

#### The web is not a DBMS

- data is mostly unstructured and untyped,
- the “correct” answer for many queries is not well-defined,
- completeness is not guaranteed,
- data manipulation guarantees are limited,
- freshness, cross-item consistency, and fault-tolerance guarantees are weaker or undefined.

#### Practical architecture

Most serious websites place a DBMS behind the web layer (for example, `facebook.com` with MySQL).

### Is ChatGPT a DBMS???

**Short answer.** No. ChatGPT is a language model interface, not a DBMS.

It can connect to database-backed systems, but by itself it does not provide the full DBMS contract (schema-centered storage, transaction guarantees, and complete declarative data management).

### 1.1.7 The scope of DBMS

The question is no longer only “*where to store bytes*”; it is how to query, update, and trust data under real workload conditions.

#### What more could we want than a file system?

- simple and efficient *ad-hoc* querying,
- robust concurrency control,
- recovery mechanisms,
- the benefits of well-designed data models.

**Ad-hoc query.** A query formed for a specific immediate need, not predefined as part of a fixed workflow. *With many queries, users, transactions, and resource constraints, complexity increases quickly. Managing that complexity is a core DBMS concern.*

#### Can the OS offer services like memory management etc? Again, not really

The operating system is essential, but it does not provide the complete data-management contract that applications need.

### 1.1.8 What is the intellectual contribution?

DBMS is not only engineering practice; it also contributes key ideas about representation, semantics, and correctness.

#### Core contributions of DBMS thinking

- **Representation of information:** data modeling,
- **Languages and systems for querying data:** complex queries with real semantics over massive data,
- **Concurrency control for data manipulation:** controlled concurrent access with transactional meaning,
- **Reliable data storage:** preserving semantics even after failures (for example, power loss).

**Transactional semantics.** The guarantee that concurrent and failing executions still behave like correct transactions under clear correctness rules.

## 1.2 DBMS Architecture

The main architectural question is simple: how do we keep one coherent meaning of data while serving different users and storing bytes efficiently on disk?

### 1.2.1 Describing data

**Data model.** A collection of concepts for describing data.

Data models are high-level abstractions: they hide low-level storage details and let us reason about structure and meaning first. Typical model families include *relational*, *hierarchical*, *graph*, and *object-oriented*.

**Relation.** A named table with rows (tuples) and columns (attributes).

**Schema.** The structural definition of a relation: attribute names, types, and organization.

In the **relational model**, data is represented as a set of records organized into relations. In **nested models**, not all data is forced into flat tables; hierarchies and arrays are natural first-class structures.

**Schema vs. data.** Schema is the type (for example, *string*); data is a value instance (for example, "hello") that follows that type.

### 1.2.2 Example: Schema of a University database

We use one running example throughout the rest of the section.

**Students**    `sid:string, name:string, login:string, age:integer, gpa:real`

**Enrolled**    `sid:string, cid:string, grade:string`

**Courses**    `cid:string, cname:string, credits:string`

Some later examples use `credits:integer`. The important point is explicit typing in the schema.

### 1.2.3 Describing data: Levels of abstraction

**External schema.** User- or application-specific views, often used for access control.

**Logical schema (conceptual level).** The main global structure seen by users and application programs.

**Physical schema (internal level).** How data is stored on disk: files, indexes, and storage layout choices.

The mapping is:

External schemas  $\rightarrow$  Logical schema  $\rightarrow$  Physical schema

Many external views map to one logical schema that is then implemented by physical structures.

## Running example across levels

*External views (access control)*

**Students\_Info**            `sid:string, name:string`

**Course\_Enrollment**   `cid:string, #enrolled:integer`

*Logical schema (what applications see)*

The logical level uses the university relations above: **Students**, **Courses**, and **Enrolled** (with `credits` often shown as `integer` in this level).

*Physical schema (how storage is organized)*

Relations can be stored as unordered files, with indexes such as `Students.sid` and `Courses.cid`.

### 1.2.4 DBMS cares about data independence types

**Data independence.** The ability to change the schema at one level of a database system without changing the schema at the next higher level.

**Logical data independence.** The ability to change the conceptual schema without changing user views.

**Physical data independence.** The ability to change the internal schema without changing the conceptual schema or user views.

This matters because data-intensive systems evolve constantly: applications should keep working while storage layouts, indexes, and conceptual structures improve over time.

### 1.2.5 Simplified DBMS architecture

The architecture has two linked flows: a *design flow* to define data and a *query flow* to access data efficiently.

#### Design flow (from requirements to storage)

- **Conceptual design:** capture the enterprise at a high level, typically with ER models.
- **Logical design:** translate ER structures into a relational model.
- **Physical design:** choose indexing and storage structures.
- **Database storage:** persist data in disk files.

#### Query flow (from SQL to results)

Applications issue SQL (rooted in relational algebra). Inside the DBMS, processing is layered:

- query optimization and execution,
- relational operators,
- access methods,
- buffer management,
- disk space management.

Cross-cutting services enforce correctness and performance across these layers: query processing, transaction management, and concurrency control.

### 1.2.6 Relational data model

In practice, relational design relies on rows/columns and explicit links between relations through keys.

**Key.** An attribute (or set of attributes) whose value identifies a tuple uniquely inside a relation.

**Foreign key.** An attribute (or set) in one relation whose values reference keys in another relation, linking the two relations.

#### Example relation instance

|                 | SID   | CID         | Grade |                 | SID   | Name  | Login      | Age | GPA |
|-----------------|-------|-------------|-------|-----------------|-------|-------|------------|-----|-----|
| <b>Enrolled</b> | 53666 | Carnatic101 | 5     | <b>Students</b> | 53666 | Jones | jones@cs   | 18  | 5.4 |
|                 | 53666 | Raggae203   | 5.5   |                 | 53688 | Smith | smith@eecs | 18  | 4.2 |
|                 | 53650 | Topology112 | 6     |                 | 53650 | Smith | smith@math | 19  | 4.8 |
|                 | 53666 | History105  | 5     |                 |       |       |            |     |     |

Here, `Enrolled.SID` acts as a foreign key referencing `Students.SID`.

This raises the next design question: *how do we design such schemas systematically?*

### 1.2.7 Database design requirements

Database design proceeds in stages, from problem understanding to implementation choices.

- **Requirements analysis:** identify user needs and what the database must support.
- **Conceptual design:** produce a high-level enterprise description (often with ER).
- **Logical design:** translate ER into the DBMS data model.
- **Schema refinement:** improve consistency and normalization.
- **Physical design:** choose indexes and disk layout.
- **Security design:** define who can access what.

### 1.2.8 What is a conceptual design

**Conceptual design.** A high-level description of the enterprise data, independent of storage and implementation details.

It answers three foundational questions:

- what are the *entities* and *relationships* in the enterprise?
- what information about them should the database store?
- which *integrity constraints* must always hold?

**ER diagram.** A pictorial representation of the conceptual schema in the ER model.

An ER diagram can be mapped into a relational schema; this bridge connects conceptual understanding to implementable database design.

### 1.2.9 Conceptual Design: Entity-Relationship Model

The ER model is the standard language for conceptual design: it captures entities, attributes, relationships, and integrity constraints before logical and physical design decisions.

#### Basics of ER model

**Entity.** A real-world object, distinguishable from other objects.

**Attribute.** A property used to describe an entity in the database.

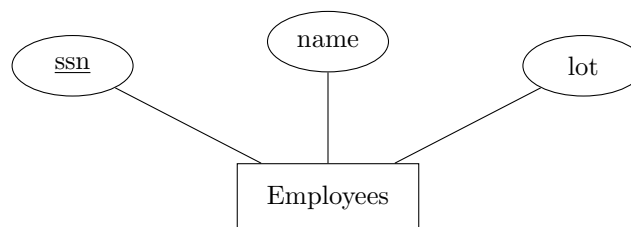
**Entity set.** A collection of similar entities that share the same attributes.

**Domain.** The set of allowed values for an attribute.

**Key (in ER).** An attribute (or minimal attribute set) that uniquely identifies entities in an entity set.

#### Entity-set schema example

For the entity set **Employees**, we use attributes **ssn**, **name**, and **lot**, with **ssn** as key.



**Instance of an entity set.** The currently stored collection of entities in that set.

Example instances for **Employees**:

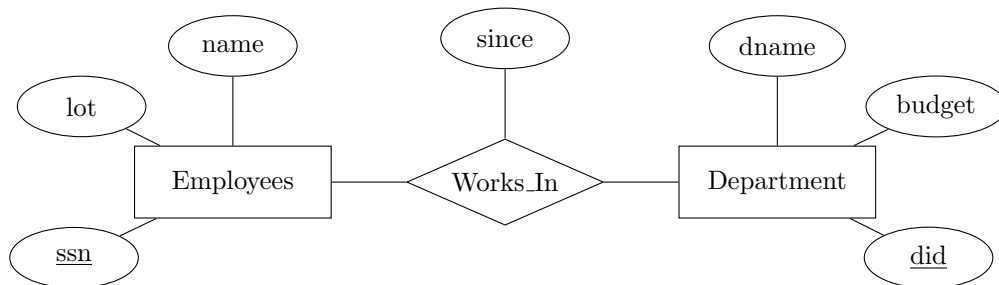
- (123-22-3666, John, P1)
- (231-31-5368, Tom, P2)

## Relationships in ER model

**Relationship.** An association among two or more entities.

**Relationship set.** A collection of similar relationships.

Relationships may have their own attributes. Example: **Works\_In** links **Employees** and **Department**, with relationship attribute **since**.



**Instance of a relationship set.** The currently stored set of relationship tuples between participating entity sets.

Example tuples for **Works\_In(ssn, did, since)**:

- (123-22-3666, 51, 1/1/91)
- (231-31-5368, 56, 3/3/93)
- (131-24-3650, 56, 2/2/92)
- (223-32-6316, 60, 3/1/92)

## Role names and repeated participation

The same entity set can participate in different relationship sets, and can also appear multiple times in one relationship set under different roles.

**Role name.** A label identifying the function of an entity set when it appears more than once in the same relationship.

Example: in **Reports\_To**, **Employees** appears twice with roles *supervisor* and *subordinate*.

